

学生学号	2025302927	大作业成绩	
------	------------	-------	--

学生大作业报告书

项目	内容
课程名称	企业级应用软件设计与开发（AI 驱动的软件开发与 Agentic AI）
课程代码	50120224001 / CS599
开课学院	计算机与人工智能学院
指导教师姓名	戚欣
学生姓名	张铭洋
学生专业	计算机技术
项目名称	StockMind：A 股多智能体投研与风险解释系统
项目方向	方向一：Agentic AI 原生开发
提交日期	2026 年 6 月 22 日

2025—2026 学年 第二学期

目录

第一部分：项目分析与设计	3
一、选题背景与设计思想	3
1.1 问题定义	3
1.2 现有方案的不足	3
1.3 项目价值	4
1.4 研究目标与研究问题	4
1.5 技术路线	4
1.6 金融安全与项目边界	5
1.7 与单模型方案对比	5
二、Specs 规格文档	6
2.1 Product Spec: 从需求描述到验收条件	6
2.2 Architecture Spec: 状态、节点与故障语义	6
2.3 API Spec: 稳定边界与可替换实现	6
2.4 Spec—Code—Test 追踪矩阵	6
三、系统架构与设计	8
3.1 核心架构	8
3.2 设计要点	8
3.3 状态流与数据流设计	8
3.4 依赖注入与双执行器	8
3.5 安全设计	8
第二部分：系统实现、调试与结果分析	9
一、关键实现与代码展示	9
1.1 AI IDE 使用记录	9
二、调试过程：实现问题、解决方法与启发	11
问题一：配置模块在导入阶段因缺少 API Key 崩溃	11
问题现象与定位	11
解决过程	11
验证与启发	11
问题二：并行专家汇聚可能导致 CIO 重复执行	11
问题现象与定位	11
解决过程	11
验证与启发	11
问题三：Python 版本差异导致状态定义无法导入	12
问题现象与定位	12
解决过程	12
验证与启发	12
问题四：真实数据能力与离线可复现性发生冲突	12
问题现象与定位	12

解决过程	12
验证与启发	12
问题五：记忆、异常与可观测性容易停留在概念层面	13
问题现象与定位	13
解决过程	13
验证与启发	13
三、测试与结果分析	14
3.1 测试策略	15
3.2 指标含义与局限	15
3.3 测试结果分析	15
四、系统升级与扩展	16
4.1 MCP 与工具生态升级	16
4.2 记忆与 RAG 升级	16
4.3 Human-in-the-loop 与事实核验	16
4.4 生产部署与 LLMOps	16
大作业小结、建议及体会	17
7.1 从“完成代码”到“交付系统”的总体变化	17
7.2 对 SDD 规格驱动开发的理解	17
7.3 对 Agentic AI 核心机制的认识	17
7.4 工程规范与生产意识	18
7.5 五次问题解决带来的方法论启发	18
7.6 AI IDE 协作中的人工责任	19
7.7 后续学习与能力演进计划	19
7.8 对课程教学的建议	19
7.9 总结与自我评价	20
参考资料与开源声明	21

第一部分：项目分析与设计

项目	内容	项目	内容
课程名称	企业级应用软件设计与开发	项目名称	StockMind 多智能体投研系统
实验者	张铭洋	专业	计算机技术
学号	2025302927	项目日期	2026 年 6 月 22 日

一、选题背景与设计思想

1.1 问题定义

股票研究是一个典型的多源异构、信息时效性强且风险敏感的复杂任务。一次相对完整的分析往往需要同时理解股票基础信息、价格与成交量变化、行业环境、公司公告、媒体新闻以及历史研究结论。不同数据源的更新频率、可靠程度和表达方式并不一致：行情数据通常是结构化数值，新闻和公告是非结构化文本，历史分析则包含大量带时间背景的判断。如果把这些信息简单拼接到同一个提示词中，大模型虽然能够生成语言流畅的回答，却未必能够清晰区分“数据事实”“模型推断”和“风险提示”，也难以说明结论来自哪一类证据。

传统股票分析软件通常将数据展示、技术指标计算与人工决策分开。它们擅长快速呈现 K 线、成交量和指标，却要求使用者自行完成跨数据源的信息整合。近年来，大语言模型为自动解读和自然语言报告生成提供了新路径，但单模型问答仍存在明显不足。第一，模型容易把多个角色的任务压缩在一次生成中，量化判断、舆情判断和风险判断缺少相互制衡；第二，模型本身没有稳定的长期记忆，同一股票在不同会话中的分析难以形成连续性；第三，调用过程通常是黑盒，出现异常时难以回答“哪个步骤失败、耗时多少、是否使用了降级数据”；第四，金融场景中的语言表达容易被误解为确定性收益承诺，因此必须把风险边界作为系统规则，而不能只依赖模型自觉。

本项目关注的核心问题不是“让大模型预测股价是否准确”，而是“如何把一个开放式股票研究任务改造成可规格化、可编排、可观察和可复现的 Agent 工程”。系统需要把复杂任务分解给职责明确的智能体，让数据获取、量化分析、舆情分析、风险控制和最终汇总形成可验证的工作流；同时还要在没有 API Key、网络不稳定或外部数据源失败时保持演示闭环。换言之，本项目的研究对象既包括 AI 能力，也包括承载 AI 能力的企业级软件工程方法。

从课程项目复现角度看，原始代码虽然已经具备 LangGraph、行情抓取和 RAG 的基本思路，但存在依赖未锁定、模块导入时立即检查密钥、缺少离线数据、缺少自动测试和缺少可观测记录等问题。这些问题使系统“在作者机器上可能运行”，却难以保证另一位评审者按照 README 能够稳定复现。因而，本项目把可复现性视为第一等质量属性：不仅要求最终报告存在，还要求每项关键功能都能由命令、测试或 Trace 证据重新验证。

1.2 现有方案的不足

第一类方案是传统的数据看板或量化脚本。它们在确定性计算、历史数据处理和图表展示方面优势明显，但对新闻语义、风险解释和跨模态证据整合支持有限。当需求从“计算某个指标”变成“综合不同证据形成一份可读研究报告”时，代码中往往出现大量条件分支和模板拼接，扩展新分析角色的成本较高。

第二类方案是直接调用单个大模型。该方案开发速度快，少量代码即可得到完整文本，但所有职责被混合在一个上下文中。模型既当数据分析师，又当新闻分析师和决策者，缺少角色隔离；任何一处输入质量问题都可能污染整份报告。单次调用也不便于设置独立超时、重试、降级和性能指标，出现错误时只能得到“调用失败”，无法定位具体业务节点。

第三类方案是完全依赖在线服务的多智能体系统。它在理想网络环境下能够展示较强能力，但课程答辩和企业演示都可能受到 API 余额、访问限速、模型服务波动以及第三方数据源变更的影响。若系统没有本地夹具数据、模型替身和确定性测试，现场展示就会变成对外部环境的赌博。工程意义上的可靠系统应当同时具备真实能力路径与可复现验证路径。

第四类方案是直接引入重型向量数据库和本地嵌入模型。它能提升语义检索能力，但也会带来模型下载、磁盘占用、版本兼容和首次启动时间等成本。对于课程 MVP，如果记忆接口尚未稳定就绑定某个具体向量数据库，反而会让业务代码与基础设施紧耦合。StockMind 因此先定义统一记忆接口，以轻量 JSONL 检索完成闭环，再把 Chroma 或 pgvector 作为明确的升级路径。

1.3 项目价值

StockMind 的直接价值是提供一条可解释的股票研究流水线。Fetcher 负责统一获取和标准化输入，量化、舆情和风险智能体分别形成独立意见，CIO 智能体只在三个专家全部完成后进行汇总。这样的结构不会自动保证结论正确，但能够显著改善责任边界：评阅者可以看到每个角色使用了什么输入、输出了什么观点，并能从 Trace 中确认 workflows 是否按设计执行。

项目的第二层价值是验证 Agentic AI 与企业级工程方法可以共同落地。AI 系统并不等于一段提示词。配置管理、状态契约、异常隔离、持久化、测试、容器化、版本控制和安全边界同样决定系统能否交付。本项目把这些工程能力放入同一个仓库，通过 Specs 将需求转化为 FR-01 至 FR-06 的可执行验收条件，使文档、代码和测试形成对应关系。

项目的第三层价值是建立“真实模式与演示模式并存”的交付策略。Demo 模式使用带 demo_fixture 标签的确定性数据和本地模型替身，不需要网络或密钥；Live 模式则连接 DeepSeek 兼容接口、Sina 实时行情和可选 Tavily 新闻。二者共享完全相同的 Agent 编排和状态结构，因此 Demo 并不是另一套临时程序，而是生产 workflow 的一种可测试依赖配置。

1.4 研究目标与研究问题

围绕上述问题，本项目设定四个研究目标。其一，构建职责分离的多智能体 workflow，验证并行专家与集中汇总的架构是否能够清晰表达复杂业务流程；其二，通过 SDD 方法把自然语言需求落实为状态字段、工具契约和验收测试，降低“文档与实现不一致”的风险；其三，建立跨会话记忆、节点追踪和异常降级机制，使系统具备基本的企业级可控性；其四，形成无密钥可复现、配置密钥可扩展的双模式交付，以应对课程演示和后续部署的不同约束。

具体研究问题包括：如何定义多个 Agent 之间最小而稳定的共享状态？如何保证三个并行专家全部完成后 CIO 只执行一次？如何让外部模型失败成为可观察的业务状态，而不是整个进程的无解释崩溃？如何在轻量实现与向量数据库扩展之间保持接口稳定？如何用自动测试验证记忆确实跨会话生效，而不仅是代码中出现了“RAG”字样？这些问题共同构成了项目实现和评估的主线。

1.5 技术路线

StockMind 将任务拆给 Fetcher、量化、舆情、风控和 CIO，多观点汇聚后才输出结论。系统以内置 Demo 保证可复现，以 Live 网关连接 DeepSeek；用检索记忆支持跨会话，用节点 Trace 支持复盘。系统只做教学研究，不做交易或收益承诺。

开发过程遵循“规格先行、接口隔离、最小闭环、逐步增强”的路线。首先编写 Product Spec，确定输入校验、多智能体、记忆、安全免责声明、可观测和离线演示六项功能要求；随后用 Architecture Spec 描述 DAG、状态流和依赖注入，用 API Spec 固定 CLI、Python API 与 Tool Contract。实现阶段先打通本地 DAG，再接入可选 LangGraph；先实现确定性 Demo，再补充 Live Adapter；先实现 JSONL 记忆，再预留向量数据库

接口。最后通过单元测试、端到端测试、Benchmark、真实 IDE 截图和 PDF 报告完成证据闭环。

技术选型上，Python 负责核心实现，LangGraph 负责正式状态图，ThreadPoolExecutor 提供本地等价并行执行，DeepSeek 兼容接口承担 Live 模型调用，Sina 与 Tavily 作为外部数据工具，JSONL 提供轻量持久化，Docker 固化运行环境。每项技术都对应具体问题，而不是为了堆叠名词：LangGraph解决状态与汇聚，工具适配解决外部能力隔离，记忆接口解决跨会话连续性，Tracing解决运行证据，Docker和环境变量解决复现与密钥安全。

1.6 金融安全与项目边界

股票研究属于高风险信息场景。本项目不连接券商账户，不执行自动下单，不根据用户资产提供个性化配置，也不承诺收益。所有 Demo 数据明确标记来源，避免把模拟行情误认为实时事实；CIO 提示词和验收测试都要求最终输出包含“不构成投资建议”。在 Live 模式中，第三方行情或新闻源失败时，状态必须标记降级来源，而不是静默伪装为真实数据。

本项目同样重视提示注入和数据污染风险。外部新闻属于不可信输入，生产系统应在进入模型前完成来源白名单、内容清洗和引用保留；记忆数据也应按用户或租户隔离，避免不同会话相互泄漏。受课程项目范围限制，本版本重点实现了来源标识、错误类型记录和免责声明，权限审计、事实核验 Agent 与人工审批节点作为后续生产化方向。

1.7 与单模型方案对比

维度	改造前：单脚本/单模型	改造后：StockMind
推理职责	混合在一个提示词	三专家独立、CIO 汇聚
状态	临时变量	TypedDict 统一契约
记忆	无或重型依赖	可替换的持久化检索接口
故障	Key 缺失即导入崩溃	延迟配置、节点降级
演示	强依赖网络	无 Key 离线闭环
评估	无	测试、Trace、Benchmark

二、Specs 规格文档

SDD 流程为“问题/边界 → 可执行验收 → 状态/API 契约 → 实现 → 自动验证”。完整规格见 product-spec.md、architecture-spec.md、api-spec.md。核心验收编号 FR-01 至 FR-06 均映射到测试或基准命令，避免文档只描述愿望而无法验证。

2.1 Product Spec: 从需求描述到验收条件

Product Spec 首先明确系统服务对象是课程研究者和评阅者，而不是实盘交易用户。用户故事分别覆盖无 Key 演示、多专家分析、运行追踪、跨会话记忆和故障降级。与普通需求文档不同，Spec 为每项能力定义可执行验收：FR-01 要求非法股票代码与倒序日期被拒绝；FR-02 要求三专家完成后 CIO 才运行；FR-03 要求同一代码第二次运行能够命中历史；FR-04 要求最终报告出现投资风险免责声明；FR-05 要求完整运行产生节点 Trace；FR-06 要求无 Key、无网络仍能完成端到端 Demo。

这些验收条件反过来约束实现。例如，如果 Demo 仍在模块导入时检查 API Key，FR-06 就不可能通过；如果记忆只完成写入而没有检索测试，FR-03 就不能声称实现；如果三个专家共享一个 report 字段，FR-02 的并行更新就存在冲突。SDD 在本项目中的作用不是增加文档数量，而是在编码之前暴露设计矛盾，让“完成”具有可检查的定义。

2.2 Architecture Spec: 状态、节点与故障语义

Architecture Spec 把业务流程表达为六节点 DAG，并定义 StockState 是唯一共享数据总线。Fetcher 只负责数据与历史上下文，三个专家只写各自报告，CIO 读取全部专家输出，Memory Store 只在最终报告就绪后执行。这种单向数据流减少了隐藏依赖，也便于在 Trace 中重建执行过程。

架构规格还定义了故障语义。配置错误在 Live 启动阶段快速失败；单个 LLM 节点异常则被隔离并转化为降级文本，允许 CIO 汇总其他证据；外部数据失败必须带来源标识；记忆存储失败不能影响报告返回。不同故障采用不同策略，体现企业系统中“并非所有异常都应该重试或终止”的思想。

2.3 API Spec: 稳定边界与可替换实现

API Spec 同时描述 CLI 与 Python Service。CLI 面向答辩和终端用户，定义 --stock-code、日期、模式和 JSON 输出；Python API 面向未来 FastAPI、桌面界面或批处理调用。Tool Contract 把基础信息、行情、新闻和记忆检索的输入输出固定下来，使 Demo Adapter、Live Adapter 以及未来 MCP Server 可以共享同一语义。

接口设计遵循“业务状态稳定、基础设施可替换”的原则。Agent 不知道行情来自 Sina 还是夹具，也不知道模型来自 DeepSeek 还是本地替身；它只依赖标准化字典和 invoke 行为。这样既降低单元测试成本，也避免外部 SDK 升级扩散到整个代码库。

2.4 Spec—Code—Test 追踪矩阵

规格	核心实现	验证证据
FR-01 输入校验	tools.validate_request	非法代码单元测试
FR-02 多智能体	graph.build_stock_graph	端到端状态断言
FR-03 长期记忆	rag.StockRAGMemory	第二次运行命中测试
FR-04 安全边界	CIO Prompt 与 Demo Model	免责声明断言
FR-05 可观测	JsonlTracer.span	Trace 数量与状态断言
FR-06 离线演示	Demo Tools/Model/Local DAG	无 Key 完整运行

三、系统架构与设计

3.1 核心架构

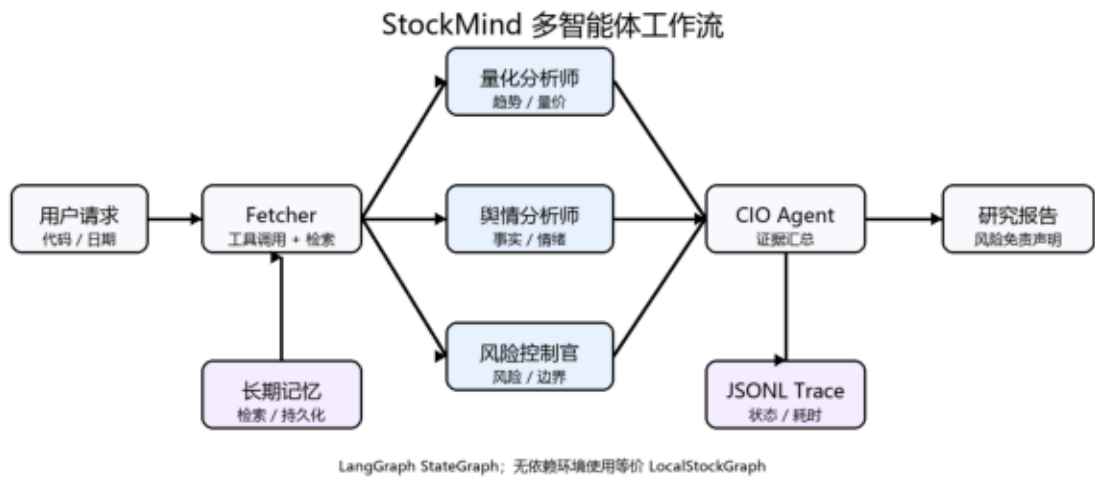


图 3-1 StockMind 多智能体核心架构

3.2 设计要点

专家节点互不写同一字段，因此适合并行；CIO 是同步汇聚点。模型、工具、记忆和 Trace 均通过依赖注入进入 Agent，便于测试和替换。生产 LangGraph 与本地 DAG 拥有相同流程语义。

3.3 状态流与数据流设计

一次请求从股票代码、开始日期和结束日期进入 Service。校验成功后生成 request ID，它既是 Trace 关联键，也可以扩展为 LangGraph thread ID。Fetcher 同步读取基础信息、行情、新闻和历史记忆，并将结果写入共享状态。此时状态中的原始证据只写一次，后续专家节点以只读方式使用，避免分析过程中修改事实输入。

量化 Agent 主要读取行情与历史上下文，舆情 Agent 读取标准化新闻，风险 Agent 读取基础信息与行情。三个节点互不依赖，可并行执行。CIO 读取三份报告和基础信息，按照摘要、证据、风险与观察清单的结构生成最终文本。最后，Memory Store 只保存必要字段，Tracer 为每个节点写入状态与耗时。该数据流把“事实获取、观点形成、决策汇总、知识沉淀”分成四个阶段。

3.4 依赖注入与双执行器

StockAgents 的构造参数包括 ChatModel、StockTools、StockRAGMemory 和 JsonlTracer。依赖由 Service 创建，使 Agent 节点保持纯业务角色。测试可以传入 Demo Model，Live 模式可以传入 DeepSeek Model，未来也可以增加 Ollama 或其他兼容模型而不修改 Graph。

正式环境优先构建 LangGraph StateGraph，最小环境则使用 LocalStockGraph。本地执行器并不是简化业务步骤，而是用标准库实现相同的 Fetch—Parallel Experts—CIO—Store 拓扑。双执行器降低了依赖安装对演示的影响，同时也要求测试关注语义一致性。后续可增加契约测试，用相同输入分别运行两种执行器并比较关键状态字段。

3.5 安全设计

安全设计包括密钥、输入、输出和日志四个层面。密钥只通过环境变量读取，.env 不进入 Git；输入必须通过格式和日期顺序校验；输出强制包含教学用途与投资风险说明；日志只保存节点、错误类型和耗时，不保存授权头和

完整提示词。对于外部新闻，当前版本保留来源字段，生产版本还应增加域名白名单、提示注入过滤和引用链接。

第二部分：系统实现、调试与结果分析

一、关键实现与代码展示

关键实现位于：src/graph.py（状态图）、src/agents.py（Agent 与降级）、src/tools.py（Function Calling 风格工具）、src/rag.py（记忆检索）、src/observability.py（Trace）。API Key 仅从 .env/环境变量读取。

1.1 AI IDE 使用记录

开发过程中使用 Trae CN 的 SOLO Agent 对多智能体 workflow 进行规格符合性审查。AI 依次读取 graph.py、Product Spec、Agent 与状态定义，对 FR-01 至 FR-06 给出逐项验证依据；本人再根据源代码和自动测试核对结论。图 4-1 展示了真实审查界面。



图 4-1 Trae CN SOLO Agent 对 Product Spec 的符合性审查

图 4-2 展示了在 Trae CN 中检查 API Spec、项目目录和测试输出的过程。规格先定义 CLI、Python API 和工具契约，再由实现与测试闭环验证，体现 SDD 的“规格—实现—验收”路径。

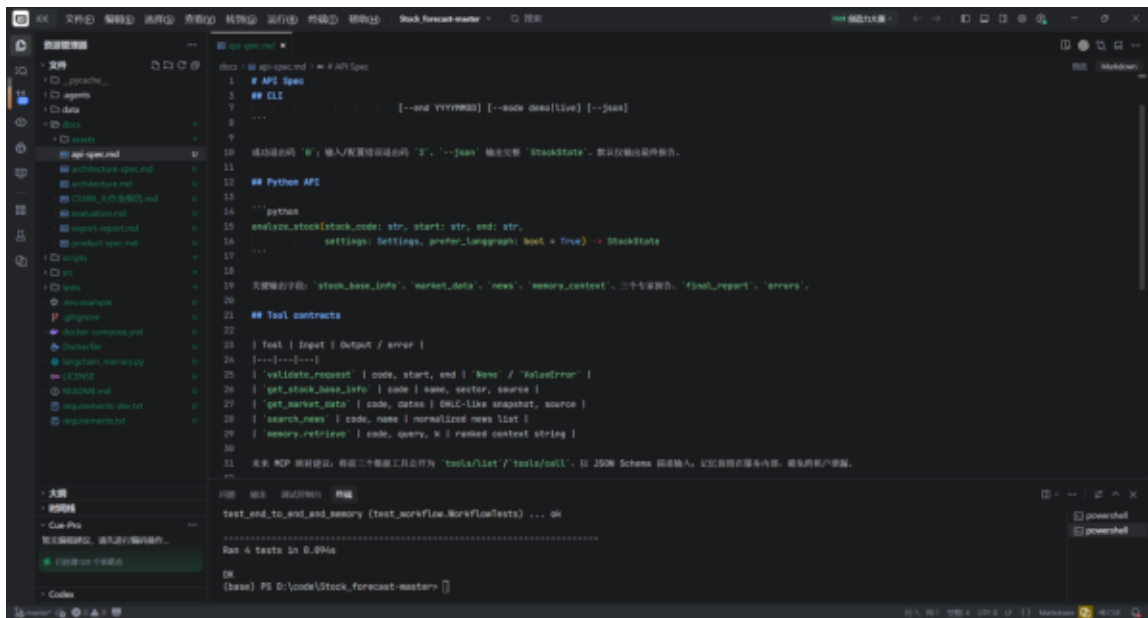


图 4-2 Trae CN 中的规格文档、工程目录与终端验证

二、调试过程：实现问题、解决方法与启发

问题一：配置模块在导入阶段因缺少 API Key 崩溃

问题现象与定位

原始项目在 config.py 被导入时立即读取 DASHSCOPE_API_KEY 和 TAVILY_API_KEY，只要任意密钥不存在就抛出异常。这个设计在作者本人配置完整的环境中不明显，但在新环境执行测试、查看帮助信息甚至仅导入某个状态模块时，都可能提前终止。它还使单元测试无法使用模型替身，因为测试文件尚未完成依赖注入，真实模型对象就已经创建。问题的本质不是“用户忘记配置 Key”，而是配置校验的生命周期与真正需要外部服务的业务时刻不一致。

解决过程

重构后，配置由 Settings.from_env() 在应用服务启动时显式加载，模块导入本身不产生网络连接和密钥副作用。系统增加 demo 与 live 两种模式：Demo 模式不要求 Key，Live 模式才执行 fail-fast 校验。模型通过 create_model(settings) 工厂创建，Agent 只依赖 ChatModel 协议，不直接依赖某个厂商 SDK。.env.example 只保留变量名称和空值，真实 .env 被 .gitignore 排除，日志也不输出密钥和完整请求头。

验证与启发

验证方式包括：在没有 .env 的环境运行完整 Demo；向 Settings 传入 Live 模式但不提供 Key，确认得到明确配置错误；扫描仓库确认不存在真实 Key。这个问题让我认识到，企业级配置管理不仅是“把 Key 放进环境变量”，还包括配置何时读取、由谁验证、错误如何呈现以及测试如何替换。延迟初始化与依赖注入能够把基础设施约束从业务逻辑中剥离，是提升可测试性和可移植性的关键。

问题二：并行专家汇聚可能导致 CIO 重复执行

问题现象与定位

多智能体结构看起来只是把 Fetcher 分别连接到量化、舆情和风控，再把三条边连接到 CIO。但在状态图框架中，“有三条入边”不一定自动等同于“等待三个节点全部完成后只执行一次”。如果错误地逐条添加从专家到 CIO 的普通边，某些调度语义可能在每个专家完成时触发一次下游节点，导致 CIO 使用不完整状态或重复生成报告。并行节点如果写入同一字段，还可能产生状态更新冲突。

解决过程

项目先在 StockState 中为三个专家设计独立字段：quant_report、news_report 和 risk_report，避免并发写冲突。正式 LangGraph 实现使用列表形式的 fan-in 边：add_edge(["quant_agent", "risk_agent", "news_agent"], "cio_agent")，把三个源节点声明为显式同步屏障。本地 LocalStockGraph 使用三个 Future 并行执行，并在逐个取得 future.result()、合并全部结果之后才调用 CIO。两种执行器虽然基础设施不同，但保持同一 DAG 语义。

验证与启发

端到端测试检查最终状态必须同时包含三份专家报告；Trace 中 CIO 事件数量应为一次；Trae CN 的规格审查也把 FR-02 对应到 fan-in 代码位置。这个问题让我理解到，多智能体系统的核心并不是 Agent 数量，而是状态一致性和调度语义。并行可以降低延迟，却会引入同步、幂等和竞争条件。绘制流程图只是设计起点，还必须把“等待谁、写什么、失败后是否继续”表达为框架能够执行的精确契约。

问题三：Python 版本差异导致状态定义无法导入

问题现象与定位

项目最初在 Python 3.10 环境通过测试，但在 Trae CN 终端使用 Anaconda Python 3.8.8 运行时，Stock State 类定义阶段出现 `TypeError: 'type' object is not subscriptable`。错误定位到 `dict[str, Any]` 和 `list[str]` 等内置泛型写法。该语法在较新 Python 中成立，在 Python 3.8 中却不能按相同方式求值。由于异常发生在导入阶段， workflow 测试被 `unittest` 标记为 `_FailedTest`，其余三个工具测试通过但端到端测试完全无法开始。

解决过程

我没有通过更换终端解释器掩盖问题，而是依据 README 宣称的运行范围修正公共状态契约。`src/state.py` 改用 `typing.Dict`、`typing.List` 与 `TypedDict`，使类型注解在 Python 3.8+ 均可解析。修复后分别用 Anaconda Python 3.8.8 和当前 Python 3.10 执行相同测试命令，两套环境均得到四项测试全部通过。真实的失败与复测输出被保留在报告截图中，展示了问题闭环。

验证与启发

这一问题说明“我的机器能运行”不等于项目可复现。Python 语法、第三方依赖和操作系统字体都可能形成隐含环境约束。兼容性不能只写在 README 中，而应由多环境测试或至少由最低版本解释器验证。更重要的是，面对 AI 生成或补全的现代语法，开发者仍需理解运行时版本边界，不能因为代码看起来类型清晰就忽略解释器实际行为。

问题四：真实数据能力与离线可复现性发生冲突

问题现象与定位

股票研究天然依赖实时行情、新闻搜索和大模型 API。如果所有节点都使用真实服务，项目更接近业务场景，却容易在答辩现场因网络、余额或限流失败；如果全部使用固定数据，演示虽然稳定，又无法说明系统具备真实扩展能力。早期代码把真实服务直接导入到 Agent 中，使这两类需求无法同时满足，也让测试结果随市场和新闻变化而变化。

解决过程

项目通过 Adapter 与依赖注入建立双模式。`DemoStockTools` 返回确定性夹具，每个字段都带 `demo_fixture` 来源标签；`DemoChatModel` 生成稳定且包含风险边界的文本。`LiveStockTools` 则访问 Sina 实时行情，并在配置 Tavily Key 后检索新闻；`DeepSeekChatModel` 通过兼容接口调用真实模型。当外部行情失败时，工具返回 `demo_fallback` 和错误类型，避免把降级数据冒充实时数据。Service 根据 Settings 选择依赖，但 Agent 和 Graph 不需要修改。

验证与启发

Benchmark 连续五次运行固定输入，验证 Demo 成功率与延迟；单元测试检查数据源必须明确标记。Live 模式则被视为能力展示路径，不用 Demo 指标替代其内容质量。这个问题带来的启发是：Mock 并不是“假的系统”，而是可控依赖边界的一种实现。只要接口、状态流和错误语义一致，本地替身能够提高测试可信度；真正需要避免的是没有来源标识、为了展示而把模拟结果宣传成实时事实。

问题五：记忆、异常与可观测性容易停留在概念层面

问题现象与定位

很多 Agent 项目在架构图中标注了 RAG、Memory 和 Trace，却没有可执行证据。例如，代码可能创建了向量数据库对象，但第二次会话是否真的检索到第一次报告并未测试；模型调用失败后可能只打印异常，最终状态看不出哪个节点降级；Tracing 若记录完整提示词和响应，还可能意外泄漏业务数据。原项目的重型 BGE 与 Chroma 方案还要求额外模型下载，不利于最小环境复现。

解决过程

本项目先定义 StockRAGMemory 接口，用 JSONL 保存请求编号、股票代码、时间和最终报告，并用轻量词元重合度完成 Top-K 检索。Fetcher 在每次分析前调用 get_stock_history_summary，Memory Store 在 CIO 之后写入本次报告。JsonlTracer.span 以上下文管理器记录 request ID、节点、状态、错误类型和耗时，不记录密钥与完整提示词。_safe_llm 捕获模型异常，把错误写入状态并返回显式降级报告，使后续 CIO 仍可基于已有证据完成输出。

验证与启发

端到端测试连续分析同一股票两次，断言第二次 memory_context 非空，并检查 Trace 至少产生预期数量的成功事件。该设计承认 JSONL 不是生产级向量数据库，但通过稳定接口保留了迁移 Chroma、pgvector 和 OpenTelemetry 的空间。我的启发是，架构先进性不能只看组件名称，更要看是否存在可观察行为和可执行验收。先用简单实现证明业务闭环，再在不破坏接口的前提下升级基础设施，通常比一开始堆叠复杂依赖更符合工程规律。

三、测试与结果分析

测试覆盖参数边界、数据来源标识、端到端 Agent DAG、记忆命中和 Trace。运行：

```
python -m unittest discover -s tests -v
```

```
python scripts/benchmark.py
```

本机实测结果为 4/4 自动测试通过；5 次离线 Demo 基准成功率 100%，平均耗时 34.26 ms，P95 为 32.56 ms。详细指标定义见 evaluation.md。

```
PS D:\code\Stock_forecast-master> python -m unittest discover -s tests -v

test_demo_data_is_labeled ... ok
test_rejects_bad_code ..... ok
test_validate_request ..... ok
test_end_to_end_and_memory . ok

Ran 4 tests in 0.091s

OK


PS D:\code\Stock_forecast-master> python scripts/benchmark.py

{"runs": 5, "success_rate": 1.0,
 "mean_latency_ms": 34.26, "p95_latency_ms": 32.56}

Evidence: reproducible automated run record / 2026-06-22
```

图 5-1 自动化测试与 Benchmark 实际运行记录（2026-06-22）

图 5-1 根据本项目实际执行结果制作，记录了 4/4 自动测试通过以及 5 次离线基准成功率 100%。它是可复现运行证据图，不冒充 Trae CN 界面截图；对应命令与原始指标均可在仓库中重新执行核验。

图 5-2 保留了真实调试轨迹：第一次运行因 Python 3.8 不支持部分内置泛型注解而失败；将 StockState 改为兼容写法后再次执行，四项测试全部通过。这一过程验证了系统的环境兼容性，也体现了通过失败证据定位并修复问题的工程方法。

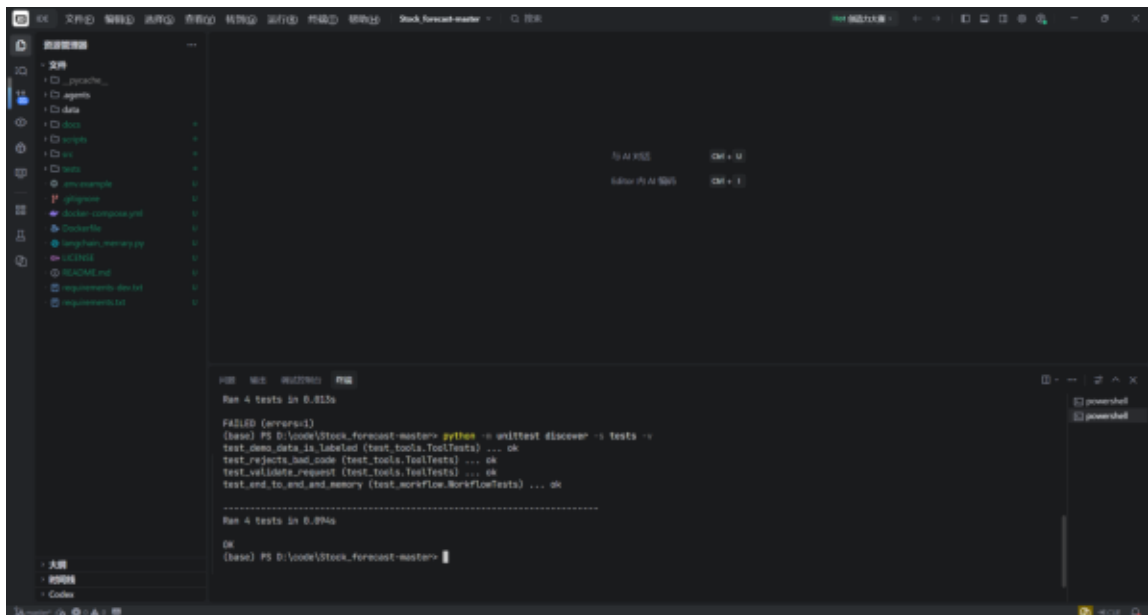


图 5-2 Trae CN 终端中的兼容性修复与 4/4 复测结果

3.1 测试策略

测试采用从小到大的三层结构。第一层是工具与边界测试，验证代码格式、日期逻辑和 Demo 数据来源；第二层是工作流测试，验证完整 DAG 能生成最终报告、错误列表为空且第二次请求命中记忆；第三层是行为 Benchmark，重复执行固定输入，观察成功率、平均耗时和 P95。三层分别回答“函数是否正确”“Agent 闭环是否成立”和“演示是否稳定”。

固定夹具使测试具有确定性。若单元测试直接访问实时行情，同一天不同时刻的数值变化都可能造成断言不稳定，也会把网络异常误判为业务错误。真实能力通过 Live Adapter 单独展示，不与离线回归测试混合。该做法借鉴企业系统对外部依赖使用 Stub/Fake 的思想。

3.2 指标含义与局限

五次 Demo 的 100% 成功率证明固定环境下工作流可以稳定闭环，约 34 ms 的平均耗时主要反映本地编排、文件记忆和模型替身开销，并不代表真实大模型响应速度。免责声明命中率和记忆命中测试验证的是行为约束，也不能替代金融结论质量。报告因此不把 Demo Benchmark 宣称为预测准确率。

Live 模式下一阶段需要增加事实一致性、引用覆盖率、风险召回率和人工可用性评价。事实一致性检查报告中的数值是否能追溯到输入；引用覆盖率衡量关键判断是否附带来源；风险召回率使用人工标注案例检查系统能否识别重大不确定性；人工评价则邀请使用者对结构清晰度、证据充分度和谨慎程度评分。所有指标应记录模型版本、提示词版本、运行日期和数据快照。

3.3 测试结果分析

四项自动测试全部通过，说明输入、Demo 来源、端到端执行和记忆回注满足当前 Spec。测试过程中发现的 Python 3.8 兼容错误被保留而非删除，它说明测试真正覆盖了环境差异。Trace 文件可以进一步用于分析节点延迟和错误分布，未来可把成功率、降级率和 P95 接入可视化看板。

四、系统升级与扩展

下一阶段按优先级：① 将 Tool Adapter 暴露为 MCP Server；② JSONL 迁移 Chroma/pgvector 并增加租户隔离；③ 增加事实核验与人工审批节点；④ FastAPI + SSE 流式界面；⑤ OpenTelemetry/LangSmith；⑥ 建立带人工标注的事实一致性与风险召回 Benchmark。若进入生产，还需采购合规行情源、权限审计、提示注入防御和灾备。

4.1 MCP 与工具生态升级

当前 Tool Adapter 已具有清晰输入输出，可进一步包装成 MCP Server，通过 tools/list 暴露能力，通过 tools/call 执行行情与新闻检索。MCP 化后，Trae、其他 Agent 或独立服务可以复用相同工具，而不需要导入 Python 项目代码。服务端还可统一处理超时、缓存、限流和审计。

4.2 记忆与 RAG 升级

JSONL 适合单用户课程 Demo，但不适合高并发和大规模语义检索。下一阶段将 StockRAGMemory 的实现替换为 Chroma 或 pgvector，使用中文嵌入模型生成向量，并按股票、用户和时间过滤。为避免历史错误不断强化，还需要增加记忆质量评分、过期策略和用户删除能力，不能把所有模型输出永久当作可信知识。

4.3 Human-in-the-loop 与事实核验

金融场景不应把模型报告直接连接交易动作。可在 CIO 之前增加 Fact Checker，对价格、新闻日期和引用链接进行二次验证；在报告发布前增加人工审批节点，允许使用者查看三个专家的分歧、修改风险级别或要求重新分析。LangGraph 的状态和条件边适合表达“自动执行—人工暂停—确认继续”的流程。

4.4 生产部署与 LLMOps

生产化可使用 FastAPI 提供 API，通过 SSE 流式返回节点进度，Docker Compose 管理服务和数据库。Tracing 迁移到 OpenTelemetry 或 LangSmith，记录节点成功率、Token 消耗、模型延迟和降级比例。安全方面还需配置密钥托管、角色权限、请求限流、审计日志和数据备份。部署前应明确行情数据授权范围和金融信息展示合规要求。

大作业小结、建议及体会

7.1 从“完成代码”到“交付系统”的总体变化

本学期《企业级应用软件设计与开发（AI 驱动的软件开发与 Agentic AI）》课程的学习，彻底打破了我传统的软件开发认知，让我完成了从被动的代码编写者到主动的智能体编排者的思维蜕变。不同于传统编程课程侧重语法学习、功能代码实现的教学模式，本课程聚焦 AI 时代软件工程的全新范式，聚焦 Agentic AI 落地实践与企业级工程思维培养，让我在技术能力、工程思想、问题解决思维上均实现了全方位成长，同时也对课程教学有了自己的思考与建议。

这次项目给我最直接的冲击是：一个程序能够输出结果，并不意味着它已经具备可交付性。最初看到原项目已经包含 LangGraph、多个分析角色和 RAG 时，我直觉上认为复现工作的重点只是安装依赖、填写 Key 和运行入口。但真正从一台新环境开始后，依赖版本、密钥生命周期、网络波动、Python 兼容、测试缺失、报告构建和 GitHub 交付连续暴露问题。我逐渐意识到，企业级开发的难点经常不在“核心算法能不能写出来”，而在系统是否能被另一个人理解、配置、运行、验证和维护。

因此，我对“完成”的定义发生了变化。过去完成一个功能，通常以界面出现结果或终端没有报错为标志；现在我更愿意追问：需求是否被规格化？异常路径是否明确？数据来源是否可追踪？无网络时能否验证？有没有测试证明行为？README 是否足以让评阅者复现？最终产物是否进入版本控制？这种问题意识比掌握某一个框架 API 更重要，它会持续影响我今后的软件工程实践。

7.2 对 SDD 规格驱动开发的理解

在个人思维与认知成长层面，我最大的收获是摆脱了“代码实现即开发终点”的固有误区。过去进行软件开发，我的核心思路是根据需求逐行编写代码、完成功能调试，关注点仅局限于代码是否能运行、功能是否可实现，缺乏对系统架构、开发规范、迭代扩展性的思考，属于典型的“面向功能编码”。而通过本课程的系统学习与期末大作业的完整实战，我深刻理解了规格驱动开发（SDD）的核心价值，明白了企业级软件开发的核心不是堆砌代码，而是先定义规范、梳理架构、明确逻辑，再通过工具与智能体落地实现。

过去我对“写文档”有一种偏见，认为文档只是代码完成后的说明材料，容易与实现脱节。此次项目中的 Product Spec 改变了这种看法。FR-01 到 FR-06 不是对代码的复述，而是在实现前规定可验收行为。例如，“支持记忆”被转化为“同一股票第二次运行时 memory_context 必须非空”；“具备可观测性”被转化为“完整运行应产生节点 Trace”；“支持离线演示”被转化为“无 API Key、无网络仍可完成闭环”。一旦需求写成这种形式，设计选择就有了清晰依据，测试也知道应该验证什么。

我还体会到 Spec 并不是一次写完的静态文件。实现过程中发现 fan-in 语义、降级来源标记和 Python 版本边界后，规格也需要补充更精确的约束。SDD 更像一个循环：先用规格减少模糊性，再由实现暴露遗漏，由测试验证并反馈规格。AI IDE 可以帮助检查 Spec 与代码的对应关系，但最终仍需要开发者判断验收条件是否真正代表业务价值。

7.3 对 Agentic AI 核心机制的认识

同时，我全面掌握了 Agentic AI 的核心底层逻辑，不再局限于只会调用 AI 工具的浅层使用模式。从前我对大模型、AI 工具的认知仅停留在代码补全、文本生成，而通过课程对 ReAct 推理机制、工具调用、记忆机制、多智能体协作的深入学习，我读懂了 AI 智能体的自主推理、状态管理与任务调度原理。在期末项目从零搭建 AI 智能体系统的过程中，我熟练运用 LangGraph 框架完成多步骤推理设计，结合检索式持久化存储实现长期记忆机制，依托 AI 原生 IDE 完成全流程开发，真正掌握了“编排智能体、让 AI 自主完成复杂工程任务”的核心能力，实现了从“手动编码”到“智能调度”的技术升级。

项目让我认识到，Agent 的“智能”不只来自模型参数，也来自外部结构。量化、舆情和风控 Agent 使用的模型可以相同，但由于角色约束、输入证据和状态字段不同，它们承担了不同职责。Fetcher 的价值不在于生成漂亮文本，而在于可靠调用工具、标准化数据和检索记忆；CIO 的价值也不在于取代专家，而在于等待全部证据后执行受约束的汇总。状态图把这些关系显式化，使复杂任务从一次不可控生成变成一系列可观察步骤。

我也不再把“多 Agent”简单理解为多调用几次模型。增加 Agent 会增加成本、延迟、状态冲突和错误传播，如果角色之间没有独立信息或清晰责任，拆分反而会让系统更复杂。本项目保留三个专家，是因为量化、舆情和风控确实具有不同证据视角；CIO 只做汇总，不重新抓取数据。这样的角色设计让我理解了智能体编排与传统微服务、工作流状态机之间的联系：都需要边界、契约和失败语义。

7.4 工程规范与生产意识

在工程实践能力上，课程让我建立了标准化、规范化的企业级开发思维。从 GitHub 仓库规范化管理、环境变量安全配置、工程目录标准化搭建，到项目分阶段迭代（Proposal-MVP-Final）、全方位测试评估与可观测性优化，整套开发流程贴合企业生产级项目标准。这让我摒弃了学生阶段随意编码、无规范、无迭代的开发陋习，深刻认识到软件工程的核心是规范、复用、迭代、可控，为今后从事 AI 软件工程、企业级系统开发相关工作奠定了扎实的工程基础。此外，通过了解 OpenClaw、Manus 等前沿智能体案例，我也及时接触了行业前沿技术动态，拓宽了技术视野，摆脱了闭门造车的学习模式。

密钥管理是一个很小却很典型的例子。以前我可能只关注“不要把 Key 写死在代码中”，现在会进一步考虑 `.env.example` 应该展示哪些变量、Live 模式何时校验、日志是否可能打印授权信息、Docker 如何注入环境变量以及 Demo 是否必须依赖密钥。类似地，可观测性也不只是多写几行 `print`，而是用统一 request ID 关联节点、记录结构化状态和耗时，同时避免 Trace 本身成为敏感信息泄漏源。

容器、测试和 Git 的意义也更加具体。Dockerfile 固化的是可复现环境；测试保存的是已经确认的行为；Git 提交保存的是可审查变更历史。它们都不是作业目录中的装饰文件，而是团队协作和长期维护的基础设施。最终通过 Public 仓库、MIT License、完整 README 和带书签 PDF 交付，让我经历了一次从代码到正式软件制品的完整流程。

7.5 五次问题解决带来的方法论启发

回顾五个实现问题，我发现有效的问题解决过程具有共同结构。第一步不是立刻修改代码，而是先缩小故障边界。例如 Key 错误发生在 `import` 阶段，说明问题属于初始化生命周期；Python 3.8 错误发生在类定义阶段，说明尚未进入工作流；推送被拒绝则是远程历史问题，而不是认证失败。只有区分“配置、语法、调度、依赖和版本控制”这些层次，解决方案才不会头痛医头。

第二步是保留可重复证据。每次修复都应该有原始错误、最小改动和复测结果。Python 兼容问题之所以值得写入报告，是因为失败截图和双版本测试形成了完整证据；记忆机制之所以可以被称为已实现，是因为端到端测试连续运行两次并断言命中。过去我可能会在修好后删除所有失败痕迹，现在认识到真实的调试链路恰恰能证明工程能力。

第三步是优先修复设计原因，而不是只绕过表面现象。缺 Key 时可以临时填一个字符串，但根因是配置在错误时间初始化；Python 版本不兼容可以换解释器，但更稳妥的做法是让公共状态契约符合声明的最低版本；网络不稳定可以反复重试，但系统层面的方案是 Demo/Live 依赖隔离。一次好的修复应该减少未来同类问题，而不仅让当前命令变绿。

第四步是承认工程权衡。JSONL 记忆没有向量数据库先进，但它让最小闭环可运行、可测试，并保留替换接口；离线 Demo 不代表真实投研质量，但它保证答辩可复现；异常降级让流程继续，却必须在结果中显式说明证据不足。这些权衡没有唯一答案，重要的是说明为什么选择、当前边界在哪里、下一阶段如何升级。

7.6 AI IDE 协作中的人工责任

Trae CN、Codex 等 AI 工具显著提高了代码阅读、规格审查和文档整理效率。AI 可以快速搜索状态字段、解释 DAG、生成测试骨架，也能帮助发现 README 与实现不一致。但这次项目同样让我看到，AI 输出必须经过人工验证。它可能默认使用较新 Python 语法，可能把“多条入边”误解为可靠同步，也可能在没有证据时给出听起来合理的性能结论。

因此，我形成了一个更谨慎的人机协作流程：先由人定义目标、边界和验收条件，再让 AI 搜索代码、提出实现或审查建议；随后由人检查依赖、运行命令、查看差异和保留证据；最后才把结果写入文档或提交 Git。对于金融免责声明、真实截图和个人课程总结等内容，更不能让生成式工具伪造。AI 是高效率的协作伙伴，但软件责任仍属于开发者。

这种协作方式也重新定义了“会编程”。未来开发者的价值不只是记住 API，而是能够提出准确问题、拆解任务、判断模型建议、设计验证并承担结果。课程提出的“从代码编写者到智能体编排者”并不意味着不再写代码，而是把编码放入更完整的需求、架构、工具和评估体系中。

7.7 后续学习与能力演进计划

下一阶段，我计划从三个方向继续深化。第一，补强 Agent 框架原理，不只会调用 LangGraph API，还要理解 checkpoint、条件边、中断恢复、幂等和并发状态合并。第二，实践 MCP 与标准化工具服务，把本项目的行情和新闻 Adapter 改造成独立 MCP Server，理解工具发现、Schema、权限和错误传播。第三，学习 LLM Ops 与评估方法，构建小规模人工标注数据集，对事实一致性、引用覆盖率和风险召回率进行真实评价。

工程方面，我希望把 StockMind 从 CLI 扩展为 FastAPI 与 SSE 服务，加入简单 Web 界面、用户会话和人工审批；将 JSONL 迁移到 pgvector，加入时间衰减与数据删除；用 OpenTelemetry 建立可观测看板；通过 GitHub Actions 在多个 Python 版本上自动运行测试。通过这些升级，我可以继续验证本课程学习到的规格、编排、评估和持续交付方法。

7.8 对课程教学的建议

基于本学期的学习体验，结合自身实战过程中的痛点，我对课程教学提出几点优化建议。首先，建议增加 Agent 核心原理的实操精讲课。课程理论内容覆盖全面，但 ReAct 推理底层逻辑、MCP 协议、多智能体协作冲突调度等前沿技术知识点难度较高，课堂理论讲解偏概括，新手落地实操时容易出现逻辑卡顿，可搭配小型随堂实操案例，帮助我们快速打通理论与实践的壁垒。

其次，建议丰富课程分层教学内容。目前课程内容兼顾理论与工程实战，但对于基础不同的同学适配性略有不足，可增设基础入门实操模块与高阶拓展模块。基础模块帮助零基础同学快速上手 AI IDE、Agent 框架使用；高阶模块可增加生产级项目部署、LLM Ops 可观测性优化、智能体性能调优等进阶内容，适配不同层次学生的学习需求。

最后，建议增加阶段性项目答疑与互评环节。课程设置了清晰的项目迭代时间节点，但学生自主开发过程中遇到的架构设计、技术选型、代码调试问题难以及时得到解答。可在每个里程碑节点后增设小型答疑课，同时增加学生项目互评环节，让大家相互借鉴优秀架构设计与工程实现思路，取长补短，进一步提升整体实践能力。

我还建议课程提供一个最小但完整的生产级参考仓库，不追求业务复杂度，而是完整展示 Spec、状态图、工具、测试、Trace、容器、CI 和部署之间的关系。学生可以在统一基线之上选择不同业务方向，减少把大量时间耗费在环境问题上的情况，同时又能比较不同 Agent 设计的差异。对于最终报告，可增加一次专门的证据审查，指导学生区分真实运行截图、自动生成示意图和未经验证的指标。

7.9 总结与自我评价

总而言之，这门课程是我研究生阶段收获最大的实践性课程之一。它不仅让我掌握了 AI 驱动软件开发、智能体系统搭建的前沿技术，更重塑了我的软件工程思维，让我精准把握了 AI 时代软件开发的转型趋势。未来，我将继续深耕 Agentic AI 领域，持续优化自身的智能体编排与企业级系统开发能力，将课程所学的工程思维落地到更多项目实践中，不断弥补自身技术短板，适应行业技术发展趋势。

如果用一句话概括本次大作业，我认为最大的成果不是生成了一份股票报告，而是建立了一套能够解释“为什么这样设计、如何证明它运行、失败后怎样处理”的开发方法。项目仍有许多不足，例如当前记忆检索较轻量、Live 内容质量尚未进行人工基准评估、Web 服务和 MCP 尚未落地，但我能够清楚描述这些边界，并为每项升级找到相对稳定的接口。这种对能力与局限的同时认识，是我从课程中获得的最重要成长之一。

参考资料与开源声明

1. LangGraph 官方文档, <https://langchain-ai.github.io/langgraph/> 2. DeepSeek API 文档, <https://api-docs.deepseek.com/> 3. 原始 Stock_forecast-master 教学代码 (本项目的改造基线)。4. 本项目使用 MIT License; 第三方依赖遵循各自许可证。AI 工具参与了代码与文档辅助, 所有内容应由提交者复核、实测并承担责任。